

practical2

BinarySearchIterative.java

```
/**
 * Performs binary search on a sorted integer array using an iterative approach.
 * A loop repeatedly narrows the search range until the target is found or the range is empty.
 * Returns the index of the target element if found, otherwise -1.
 */
public class BinarySearchIterative {

    /**
     * Iteratively searches for the target value in the sorted array.
     *
     * @param arr    the sorted array to search in
     * @param target the value to search for
     * @return the index of the target, or -1 if not found
     */
    public static int binarySearch(int[] arr, int target) {
        // The left boundary of the current search range: start at the first index (0).
        int left = 0;
        // The right boundary of the current search range: start at the last index.
        int right = arr.length - 1;

        // Keep searching as long as the search range is still valid (left has not passed right).
        while (left <= right) {
            // Find the middle index of the current search range.
            // Using "left + (right - left) / 2" instead of "(left + right) / 2" avoids integer overflow.
            int mid = left + (right - left) / 2;

            // If the middle element equals the target, we have found it: return its index.
            if (arr[mid] == target)
                return mid;

            // Discard the half that cannot contain the target:
            // If the middle value is smaller than the target, the target must be in the right half,
            // so move the left boundary just past the middle element (search the right half).
            if (arr[mid] < target)
                left = mid + 1;
            else
                // Otherwise the target must be in the left half,
                // so move the right boundary just before the middle element (search the left half).
                right = mid - 1;
        }

        // If the loop ends, the search space is empty and the target was never found, so return -1.
        return -1;
    }

    public static void main(String[] args) {
        // Create the sorted array that will be searched.
        int[] arr = {5, 10, 15, 20, 25, 30, 35};
        // The value to search for inside the array.
        int target = 25;

        // Call the iterative binary search on the whole array and store the result.
        int result = binarySearch(arr, target);

        // If the returned value is not -1, the target was found, so print its index.
        if (result != -1)
```

```
        System.out.println("Element found at index: " + result);
    else
        // Otherwise the target was not found, so print a message saying so.
        System.out.println("Element not found");
    }
}
```

BinarySearchRecursive.java

```
/**
 * Performs binary search on a sorted integer array using a recursive approach.
 * Each call halves the search range until the target is found or the range is empty.
 * Returns the index of the target element if found, otherwise -1.
 */
public class BinarySearchRecursive {

    /**
     * Recursively searches for the target value within the given range.
     *
     * @param arr    the sorted array to search in
     * @param left   the left boundary index (inclusive)
     * @param right  the right boundary index (inclusive)
     * @param target the value to search for
     * @return the index of the target, or -1 if not found
     */
    public static int binarySearch(int[] arr, int left, int right, int target) {
        // Base case: the search space is exhausted (left has moved past right),
        // which means the target is not present anywhere in the array, so return -1.
        if (left > right)
            return -1;

        // Find the middle index of the current search range.
        // Written as "left + (right - left) / 2" instead of "(left + right) / 2"
        // to prevent integer overflow when left and right are both very large.
        int mid = left + (right - left) / 2;

        // If the middle element equals the target, we have found it: return its index.
        if (arr[mid] == target)
            return mid;

        // Recurse into the appropriate half of the array based on the comparison:
        // If the middle value is greater than the target, the target (if present)
        // must lie in the left half, so search only the indexes before the middle.
        if (arr[mid] > target)
            return binarySearch(arr, left, mid - 1, target);

        // Otherwise (middle value is smaller than the target) the target (if present)
        // must lie in the right half, so search only the indexes after the middle.
        return binarySearch(arr, mid + 1, right, target);
    }

    public static void main(String[] args) {
        // Create the array to search in. It must be sorted because binary search
        // only works correctly when the elements are in ascending order.
        int[] arr = {5, 10, 15, 20, 25, 30, 35};
        // The value we are looking for inside the array.
        int target = 30;

        // Call the recursive binary search over the whole array, starting at index 0
        // and ending at the last index (arr.length - 1). Store the returned index.
        int result = binarySearch(arr, 0, arr.length - 1, target);

        // If the returned value is not -1, the target was found, so print its index.
        if (result != -1)
            System.out.println("Element found at index: " + result);
        else
            // Otherwise (-1) the target was not found, so print a message saying so.
            System.out.println("Element not found");
    }
}
```


BubbleSort.java

```
// Import Arrays: provides the toString() helper used to print the sorted array nicely.
import java.util.Arrays;

/**
 * Sorts an integer array using the Bubble Sort algorithm.
 * Includes an early-exit optimization: if no swaps occur during a pass,
 * the array is already sorted and the algorithm terminates early.
 */
public class BubbleSort {

    /**
     * Sorts the given array in-place using bubble sort.
     *
     * @param arr the array to sort
     */
    public static void bubbleSort(int[] arr) {
        // n holds the size of the array so we know how many elements to compare.
        int n = arr.length;

        // Outer loop: number of passes required
        // Each pass moves the largest remaining element to its correct end position.
        for (int i = 0; i < n - 1; i++) {
            // swapped tracks whether any swap happened during this pass, reset for each pass.
            boolean swapped = false;

            // Inner loop: compare adjacent elements
            // Each pass shrinks the unsorted part by one (the last i elements are already placed).
            for (int j = 0; j < n - i - 1; j++) {
                // If the left element is bigger than the right, they are out of order.
                if (arr[j] > arr[j + 1]) {
                    // Swap if elements are in wrong order
                    // Save the left value before overwriting it.
                    int temp = arr[j];
                    // Move the smaller right value into the left slot.
                    arr[j] = arr[j + 1];
                    // Move the larger value into the right slot.
                    arr[j + 1] = temp;
                    // Record that a swap happened, so the outer loop knows sorting is still needed.
                    swapped = true;
                }
            }

            // If no swaps occurred, the array is already sorted
            if (!swapped)
                // Exit the outer loop early since the array is now sorted.
                break;
        }
    }

    public static void main(String[] args) {
        // Create a sample unsorted array to demonstrate the sorting.
        int[] arr = {64, 34, 25, 12, 22, 11, 90};

        // Call the bubble sort method which sorts the array in-place.
        bubbleSort(arr);

        // Print a heading before the sorted values.
        System.out.println("Sorted Array:");
        // Print the whole sorted array on one line using the Arrays helper.
    }
}
```

```
        System.out.println(Arrays.toString(arr));  
    }  
}
```

InsertionSort.java

```
// Import Arrays: provides the toString() helper used to print the sorted array nicely.
import java.util.Arrays;

/**
 * Sorts an integer array using the Insertion Sort algorithm.
 * Builds the sorted array one element at a time by shifting elements
 * to insert each new element into its correct position.
 */
public class InsertionSort {

    /**
     * Sorts the given array in-place using insertion sort.
     *
     * @param arr the array to sort
     */
    public static void insertionSort(int[] arr) {
        // n holds the size of the array so we know how many elements to sort.
        int n = arr.length;

        // Start from the second element (first element is trivially sorted)
        // i tells us which element we are inserting into the already-sorted prefix.
        for (int i = 1; i < n; i++) {
            int key = arr[i]; // Element to be positioned
            // j starts at the element just before the key and moves leftwards.
            int j = i - 1;

            // Shift elements greater than key one position to the right
            // Keep shifting while j is still valid and the current element is bigger than the key.
            while (j >= 0 && arr[j] > key) {
                // Move the bigger element one slot to the right to make room.
                arr[j + 1] = arr[j];
                // Step backwards to check the next element on the left.
                j--;
            }

            // Place the key in its correct position
            // The loop stopped after the last bigger element, so put the key in the open slot.
            arr[j + 1] = key;
        }
    }

    public static void main(String[] args) {
        // Create a sample unsorted array to demonstrate the sorting.
        int[] arr = {12, 11, 13, 5, 6};

        // Call the insertion sort method which sorts the array in-place.
        insertionSort(arr);

        // Print a heading before the sorted values.
        System.out.println("Sorted Array:");
        // Print the whole sorted array on one line using the Arrays helper.
        System.out.println(Arrays.toString(arr));
    }
}
```

LinearSearch.java

```
/**
 * Searches for a target value in an integer array using Linear Search.
 * Traverses the array sequentially, checking every element one by one in order,
 * and returns the index of the first occurrence, or -1 if the target is not found.
 */
public class LinearSearch {

    /**
     * Performs linear search on the given array.
     *
     * @param arr    the array to search in
     * @param target the value to search for
     * @return the index of the target, or -1 if not found
     */
    public static int linearSearch(int[] arr, int target) {
        // Loop through every index of the array, starting from the first element.
        for (int i = 0; i < arr.length; i++) {
            // Check whether the current element equals the value we are searching for.
            if (arr[i] == target) {
                // Match found: return the current index immediately.
                return i;
            }
        }
        // If the whole array was checked with no match, the target is not present, so return -1.
        return -1;
    }

    public static void main(String[] args) {
        // The unsorted array to be searched.
        int[] arr = {12, 45, 7, 23, 89, 34};
        // The value we want to find inside the array.
        int target = 23;

        // Perform the linear search and store the returned index (or -1 if not found).
        int result = linearSearch(arr, target);

        // If the result is not -1, the target was found at that index.
        if (result != -1) {
            System.out.println("Element found at index: " + result);
        } else {
            // Otherwise the target was not found anywhere in the array.
            System.out.println("Element not found");
        }
    }
}
```

MergeSort.java

```
// Import Arrays: provides the Arrays.toString() utility used to print the array contents in a readable format.
import java.util.Arrays;

/**
 * Sorts an integer array using the Merge Sort algorithm (divide and conquer).
 * Recursively splits the array into halves, sorts each half, and merges them.
 */
public class MergeSort {

    /**
     * Merges two sorted subarrays (arr[left..mid] and arr[mid+1..right])
     * into a single sorted subarray.
     */
    public static void merge(int[] arr, int left, int mid, int right) {
        // Work out the size of the left subarray (indexes left..mid).
        int n1 = mid - left + 1;
        // Work out the size of the right subarray (indexes mid+1..right).
        int n2 = right - mid;

        // Create two temporary arrays: L will hold the left half and R will hold the right half.
        int[] L = new int[n1];
        int[] R = new int[n2];

        // Copy the elements of the left half of arr into the temporary array L.
        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        // Copy the elements of the right half of arr into the temporary array R.
        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        // i and j are the read positions for L and R; k is the write position back into arr.
        int i = 0, j = 0;
        int k = left;

        // Merge the two sorted halves back into arr by repeatedly picking the smaller front element.
        while (i < n1 && j < n2) {
            // If the front of L is smaller than or equal to the front of R, take it.
            if (L[i] <= R[j]) {
                arr[k] = L[i];
                i++; // Move to the next element of L.
            } else {
                // Otherwise take the front of R instead.
                arr[k] = R[j];
                j++; // Move to the next element of R.
            }
            // Move the write position forward in arr.
            k++;
        }

        // One of the halves may still have leftover elements; copy any remaining elements of L.
        while (i < n1) {
            arr[k] = L[i];
            i++;
            k++;
        }

        // Copy any remaining elements of R.
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }
}
```

```

        j++;
        k++;
    }
}

/**
 * Recursively sorts the subarray from left to right using merge sort.
 */
public static void mergeSort(int[] arr, int left, int right) {
    // Only continue if the current segment has more than one element (left is still before right).
    if (left < right) {
        // Find the middle index so the segment can be split into two halves.
        // This form of the expression avoids integer overflow.
        int mid = left + (right - left) / 2;

        // Recursively sort the left half (left..mid).
        mergeSort(arr, left, mid);
        // Recursively sort the right half (mid+1..right).
        mergeSort(arr, mid + 1, right);

        // Both halves are now sorted, so merge them back together in sorted order.
        merge(arr, left, mid, right);
    }
}

public static void main(String[] args) {
    // Create an unsorted array that will be sorted using merge sort.
    int[] arr = {38, 27, 43, 3, 9, 82, 10};

    // Call merge sort on the whole array, from index 0 to the very last index.
    mergeSort(arr, 0, arr.length - 1);

    // Print a heading before showing the sorted array.
    System.out.println("Sorted Array:");
    // Arrays.toString() converts the array into a readable [a, b, c] format for printing.
    System.out.println(Arrays.toString(arr));
}
}

```

SelectionSort.java

```
// Import Arrays: provides the Arrays.toString() utility used to print the array contents in a readable format.
import java.util.Arrays;

/**
 * Sorts an integer array using the Selection Sort algorithm.
 * Repeatedly finds the minimum element from the unsorted portion
 * and swaps it into the correct position.
 */
public class SelectionSort {

    /**
     * Sorts the given array in-place using selection sort.
     *
     * @param arr the array to sort
     */
    public static void selectionSort(int[] arr) {
        // Store the length of the array once so we do not recompute it inside the loops.
        int n = arr.length;

        // Move the boundary of the unsorted subarray one element at a time
        // Outer loop: everything before index i is already sorted; everything from i onwards is unsorted.
        for (int i = 0; i < n - 1; i++) {
            // Assume the first element of the unsorted portion is the smallest so far.
            int minIndex = i;

            // Find the minimum element in the unsorted portion
            // Inner loop: scan the rest of the unsorted portion looking for a smaller element.
            for (int j = i + 1; j < n; j++) {
                // If we find an element smaller than the current minimum, remember its index.
                if (arr[j] < arr[minIndex])
                    minIndex = j;
            }

            // Swap the found minimum with the first element of the unsorted portion,
            // which places it in its correct sorted position.
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }

    public static void main(String[] args) {
        // Create an unsorted array that will be sorted using selection sort.
        int[] arr = {29, 10, 14, 37, 13};

        // Sort the array in-place (the original array itself gets rearranged).
        selectionSort(arr);

        // Print a heading before showing the sorted array.
        System.out.println("Sorted Array:");
        // Arrays.toString() converts the array into a readable [a, b, c] format for printing.
        System.out.println(Arrays.toString(arr));
    }
}
```

TwoDArraySearch.java

```
// Import Scanner: used to read text input typed on the keyboard.
import java.util.Scanner;

/**
 * Reads a 2D array and a target number from the user, then searches for
 * the target in the array and prints its coordinates if found.
 */
public class TwoDArraySearch {
    public static void main(String[] args) {
        // Create a Scanner object that reads input typed on the keyboard.
        Scanner sc = new Scanner(System.in);

        // Ask the user how many rows the 2D array should have and read the number.
        System.out.print("Enter number of rows: ");
        int rows = sc.nextInt();

        // Ask the user how many columns the 2D array should have and read the number.
        System.out.print("Enter number of columns: ");
        int cols = sc.nextInt();

        // Create a 2D array with the given number of rows and columns.
        int[][] arr = new int[rows][cols];

        // Print a message asking the user to start entering the array elements.
        System.out.println("Enter the elements of the 2D array:");
        // Outer loop: go through each row of the 2D array.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < cols; j++) {
                // Read the next number typed and store it at position [i][j].
                arr[i][j] = sc.nextInt();
            }
        }

        // Ask the user which number they want to search for and read it.
        System.out.print("Enter the number to search: ");
        int target = sc.nextInt();

        // Linear search through each element of the 2D array
        // A flag that remembers whether the target has been found anywhere yet (starts false).
        boolean found = false;
        // Outer loop: go through each row while searching.
        for (int i = 0; i < rows; i++) {
            // Inner loop: go through each column of the current row.
            for (int j = 0; j < cols; j++) {
                // Check whether the element at (i, j) matches the target number.
                if (arr[i][j] == target) {
                    // Match found: print the row and column coordinates of the match.
                    System.out.println("Number found at index: (" + i + ", " + j + ")");
                    // Mark the flag as true so we know a match has been found.
                    found = true;
                    // Stop searching the inner loop (columns) for this row.
                    break;
                }
            }
        }
        // If the target was already found, stop searching the outer loop (rows) too.
        if (found) {
            break;
        }
    }
}
```

```
}

// If the flag is still false, the target was never found in the whole array.
if (!found) {
    System.out.println("Number not found in the array.");
}

// Close the Scanner to free the system resources it uses.
sc.close();
}
}
```